

EECS 482

Lab 0: Introduction and Tools

Agenda

- Administrative Things
- Git/GitHub
- GDB
- C++ Review
 - Project Structure/ODR
 - Static keyword
 - Smart pointers

— — —

Please do the following

— — —

1. Submit your picture
os.eecs.umich.edu > “Update Your Student Information”
2. Add username to your Piazza display name
Eg FIRST LAST (UNIQUE)
[Piazza](#) >  > “Account/Email Settings”
3. Complete [Project 0](#)

Project 0

1. Link your github account to the EECS 482 organization; your group will be provided private github repositories for each project.
2. You must use VSCode or Neovim to edit your projects.
 - a. Enable the version482 extension/plugin – this takes periodic **diffs** of your projects, like Google Docs (feel free to check out the source code!)
3. Set up an SSH key to connect to Github without authenticating every time

Structure of Lab

— — —

- Synchronous, streamed, recorded
- Every Friday
- Lecture and project material
- We may do in-lab exercises together
- Slides will be posted a few days ahead of time

Piazza

— — —

- A quick way to get your questions answered
- Helps the rest of your classmates too!
- But **before asking**, please make a conscious effort to check whether your question has already been answered:
 - *The spec*
 - Other Piazza posts
 - Man pages
 - Your favorite search engine
 - cppreference
 - Stack Overflow
- If your question involves specific design, code, or test cases, it must be private

Office Hours

- <https://os.eecs.umich.edu/oh.php>
- In-Person (**encouraged**)
 - BBB: Put your location and what you're wearing. We will come to you
- On-line
 - Put a Zoom or Google Meet link in the location field
 - If your meeting has a password, include that too
- Pay attention to your place on the queue
 - If no one is in your meeting link, you may be popped
 - The queue may go slightly out of order
- Please come prepared with specific questions
- Office Hours is a good place to go for:
 - Project, lecture, lab

Office Hours Continued

- Office hours will be increased the week project is due
- We try to empty the queue (but may need to close it to new requests if the queue is too long)
- Come prepared to explain what you've tried
 - Test cases you've written
 - Efforts to find bugs with GDB
 - Spreadsheet?
- In general, we won't tell you how to design your project, find bugs for you, or give you hints on autograder tests.
- Help us help you!

Projects

- The bread and butter of EECS 482
- Projects 2-4 require a group of 2-3
 - Declare your group by May 28, 2025
- Projects are also hand-graded
 - Style, efficiency, design, safety
 - Bugs that the autograder cannot catch
 - Use good coding paradigms, avoid code duplication, add good/useful comments

Project 1: 5%

Project 2: 15%

Project 3: 15%

Project 4: 15%

What makes the projects hard?

- You develop from scratch to solve open-ended problems
- The bugs you will encounter are very tricky/subtle
- The specs are thorough; read them carefully to make sure you understand and meet **every** requirement
- You are limited to viewing 1 submit per day
 - **You** must thoroughly test your code; the AG is **not** a debugging tool

Soft Deadlines

- Handgrading will be based on the highest scoring submit before the due date
- Autograder will remain open until August 13th, but will not allow increased points on hand grading
- You are expected to know the respective project knowledge prior to the midterm and final
- **Please read syllabus closely** to understand this policy

Project tips

- Start early
- Think first, code later
 - Strive for a good design before writing a single line
 - Read the spec thoroughly to ensure you cover all bases
- Code with memory safety, style, and simplicity in mind
 - Will save you many hours of tedious debugging
- Write targeted tests
 - Incrementally test subsystems before the entire project is complete
 - Deliberately write tests to break your code
 - Try shrinking test cases to be more targeted
- Avoid premature optimization
- Consult Piazza for questions!

Git/GitHub

Git refresher

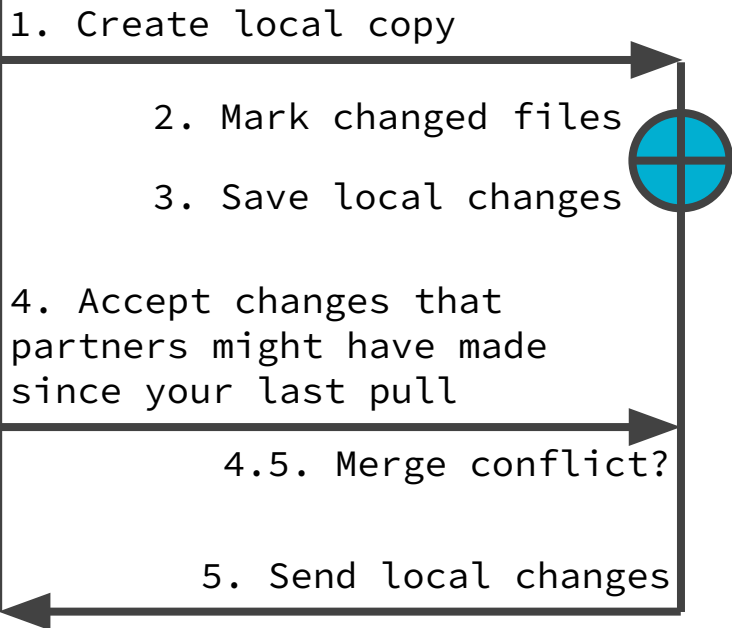
1. `git clone <repo link>`
2. `git add <filename>`
3. `git commit [-m]`
4. `git pull`
5. `git push`

Always pull before you push!

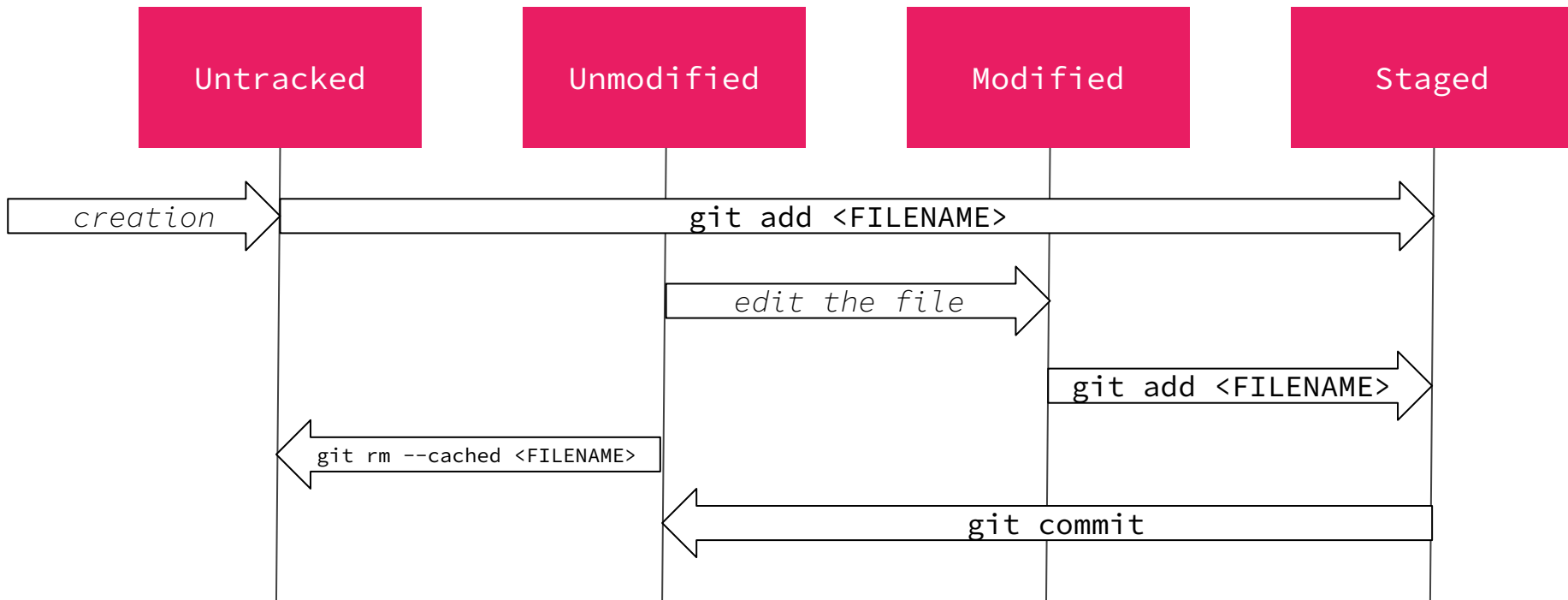
Remote
repo



Local
repo



Lifecycle of a file in a git repo



Pushing to GitHub without authenticating

Set up an SSH key so that you can interface with GitHub through the terminal without inputting your password – the Makefiles provided to you with the projects require that you do so!

Instructions:

- [Set up SSH key](#)
- [Link SSH key to UM SSO](#)

Git Tip:

Commit often with small
commits.

This makes it easier to
see where a bug may have
been introduced.

— — —

More Git Tips

- Using a **.gitignore** file can declutter your repo (usually files that can be generated should be ignored, eg. object files)
- Effectively using **git branch** can simplify collaboration
- As one of the most ubiquitous developer tools, you can probably find an answer to your exact Git issue online
- Please do not delete **./autotag.sh push** in the Makefile, but you may change it to just **./autotag.sh** to decrease compilation time (482 specific)

Tracking Group Contributions

If we are in doubt about a group's distribution of work, we may check the git commit logs to see who is doing work.

If coding as a group, switch GitHub accounts every so often:

```
$ git config --get user.name
Kelly Goss
$ git config user.name "Scott Brinley"
$ git config user.email "sbrinley@umich.edu"
```

For a single commit, you can also append
`Co-authored by: Scott Brinley <sbrinley@umich.edu>`
to the commit message. [Github supports co-authors](#).

EECS 482 Git/GitHub Tutorial

— — —

Resources

- [Syllabus](#)
- [Piazza](#)
- Office hour queues: [EECS 482](#), [EECS 498-002](#)
- [FAQ for EECS 498-002](#)
- [C++ reference](#)
- Git: [EECS 482 tutorial](#), [reference](#), [book](#).
- [Beej's Guide to Network Programming](#)

`https://os.eecs.umich.edu/git-tutorial/`

CAEN

CAEN

Helpful debugging environment

Nearly the same environment as auto grader

Login: `ssh <uniquename>@login.engin.umich.edu`

*If off campus you need to use the VPN!

Connecting through your IDE

You can do this with nearly all modern IDEs!

See @7 for setup information.



Remote - SSH

Microsoft  microsoft.com |  24,687,442 |      (191)

Open any folder on a remote machine using SSH and take advantage of VS Code's full feature ...

Disable

Uninstall 

Switch to Pre-Release Version

 Auto Update 

GDB

GDB and core dumps

What are they?

GDB is a command-line debugger available on CAEN. (**LLDB** is basically the same thing on Mac)

Core dumps/memory dumps are a snapshot of a program's state, usually taken just as the program crashed. YOU CAN SEE WHAT HAPPENED DURING THE CRASH (and usually figure out why!)

Learn this because:

GDB is ubiquitous. On any system, even if it's infeasible to hook in with the VS or Xcode debugger, you'll have GDB.

Staff are all familiar with GDB.

Core dumps give you a way to debug non-deterministic (i.e. hard to reproduce) bugs in your code.

GDB: basic usage

— — —

1. **Compile** your code with **debugging symbols** (i.e. with `-g`, or `-g3`)
2. **Load** the executable into GDB:
 - `$ gdb ./your_project.exe`
3. Set **breakpoints** on lines you want to inspect.
 - `gdb> break foo.cpp:25`
 - `gdb> break main()`
 - `gdb> break 25`
 - `gdb> break MyClass::bar`
4. **Run** the executable.
 - `gdb> run`
 - `gdb> run --arg 0.3 > output`
 - `gdb> r 0.3 > output`
5. Wait for the executable to **break** (on a breakpoint, or by hitting CTRL-C).
6. **Inspect** what's going on.
 - `gdb> bt`
 - `gdb> p my_var`
 - `gdb> list`
 - `gdb> next`
 - `gdb> continue`

GDB: basic command cheat sheet

- **b** *location* (set breakpoint)
 - `b main()`
 - `b main.cpp:12`
 - `b MyClass::~~MyClass()`
- **dis** *n* (disable breakpoint *n*)
 - `dis <breakpoint_num>`
- **en** *n* (enable breakpoint *n*)
- **del** *n* (delete breakpoint *n*)
- **r** *args* (run or restart)
 - `r --num_cpus 4`
- **kill** (kill)
- **s** *n* (step *n* times)
 - Steps *into* functions
- **n** *n* (next *n* times)
 - Steps *over* functions
- **c** (continue)
 - Continue until end or next breakpoint

GDB: basic command cheat sheet

- **p** *expr* (print)
 - Print value of variable or expression
 - Works on pretty much anything
- **i** *topic* (info)
 - **i b** (list breakpoints)
 - **i thr** (list threads)
- **f** (frame)
 - View current line
- **list** (list code)
 - View nearby code lines
- **bt** (backtrace)
 - View the call stack
- **up** *n* (up *n* frames)
 - less recent frames
- **down** *n* (down *n* frames)
 - More recent frames
- **thr** *n* (switch to thread *n*)
 - **thr 3**

Pro Tip: Conditional breakpoints!

- **cond** *n* *expr*
 - Sets a condition on breakpoint *n*
 - If no *expr*, gets rid of the condition for breakpoint *n*

Example for a for loop:

```
(gdb) $ cond 1 (i >= 4)
```

Pro Tip: Automatic breakpoint commands!

Allow you to perform specific actions every time a breakpoint is hit.

```
(gdb) $ break main.cpp:44
```

```
(gdb) $ commands
```

```
> backtrace
```

```
> print var
```

```
> end
```

Core dumps

- Core dumps are *files* created when a program crashes, populated with the *state of the program's memory* at the instant the crash occurred.
 - Usually named something like `core` or `core.12345`, where 12345 was the Process ID (PID) of the program you ran.
- Instead of *running* a program in GDB, you can tell GDB to load your core dump
- When viewing a core dump, you can do everything you can normally do in GDB that doesn't involve running the program (e.g. `step`, `next`, etc.)

Creating core dumps

- Create core dumps by running:

```
$ ulimit -c unlimited
```

```
$ ./your_program.exe
```

```
Segmentation fault (core dumped)
```

- View core dumps in GDB by running:

```
$ gdb ./your_program.exe -c core.12345
```

- Core dumps can be created in a variety of locations; Google your OS to figure it out

Note that your program needs to *crash* to produce a core dump.

An easy way to ensure this is with `assert()` statements.

Getting core on CAEN

- Generated coredumps appear in `/var/lib/systemd/coredump/` as an *lz4 compressed file*
- To uncompress
 - `$ unlz4 /var/lib/systemd/coredump/core.PROGRAM.lz4 OUTPUT-FILENAME`
 - See ``$ man lz4`` for more information on unlz4
 - Note that ``unlz4`` without the OUTPUT-FILENAME argument will fail trying to modify the existing compressed coredump, while you only have read permissions
 - CAEN uses Access Control Lists- see ``$ man getfacl``
- Then proceed with the uncompressed coredump as usual

Full GDB cheat sheet

— — —

<http://users.ece.utexas.edu/~adnan/gdb-refcard.pdf>

GDB Exercise

https://github.com/eecs482/gdb_demo

C++ Project Structure

Multiple Sources

- C++ programs can have a large number of source files
- Each file corresponds to a module – a piece of code that forms a logical unit of some sort
- A module might want to use symbols from other modules
- How do we structure a C++ project to deal with this complexity?

Declaration vs Definition

- A **declaration** tells the compiler that a symbol exists
- The **definition** tells the compiler how that symbol is implemented
- General rule: Put declarations in **header files (.h/.hpp)**, and definitions in **source files (.c/.cpp/.cc)**
 - Modules which want to use another module's symbols can **#include** that other module's header file

Declaration vs Definition: Example

```
// math.h  
  
// Declares that the symbol add exists  
  
double add(double a, double b);
```

```
// math.cpp  
  
// Defines the implementation of add  
  
double add(double a, double b) {  
    return a + b;  
}
```

- Now, files which want to use add will **#include "math.h"**
- After "math.h" is included, the includer will know about the symbol named add, and the **linker** will link together this symbol with the correct definition in math.cpp

One-Definition Rule (ODR)

- **One-Definition Rule (ODR):** it is an error to define twice a struct or class of the same name, and it is an error to define twice a function (in C) of same name or a function of same signature (in C++)!
- **Thus, never ever #include .cpp files**
- OK to have multiple *declarations*, which is why we can include the same header file from multiple locations.
 - This is also why we want to put *declarations in header files* and *definitions in source files*
 - Function definitions in header files are only safe if they're of inline functions or function templates

Extern

- The Declaration vs Definition example above used a function
- ODR and the same concepts apply to **global variables** and **classes** also
- Particularly, a common pitfall is to violate ODR by defining a global variable more than once
- Say you want a global map. What is the problem with the following?

```
// classroom.h
#include <unordered_map>
std::unordered_map<std::string, std::string> students;
```

Extern

```
// class.h
#include <unordered_map>

std::unordered_map<std::string, std::string> students;
```

- If multiple files include class.h, ODR will be violated!
- This is because **students** is a *defining* declaration in the code above.
 - You are declaring it so that other files know what **students** is, but you are also defining it at the same time!
- How can we declare a global variable without defining it?

Extern

- Make the declaration a *referencing* declaration using the **extern** keyword and define the variable in a .cpp file
 - .cpp files should never be included by another file, so you can rest assured that the definition will only occur once in your program

```
// class.h  
  
#include <unordered_map>  
  
extern std::unordered_map<std::string, std::string> students; // declaration
```

```
// class.cpp  
  
#include "classroom.h"  
  
std::unordered_map<std::string, std::string> students; // definition
```

References/Resources

- http://websites.umich.edu/~eecs381/lecture/MultipleSources_Linker.pdf

Static Variables and Methods

static keyword

Within a class

- A static variable is shared between all instances of the class
- A static function must only use static member variables and call static functions, and can be invoked independent of a class instance

Example: Global Counter

```
class foo {  
    public static int counter = 0;  
    public static void inc(){  
        counter += 1;  
    }  
}
```

```
int foo::counter = 0;  
foo a;  
foo b;
```

```
a.inc();  
b.inc();
```

```
cout << foo::counter; //2  
foo::inc();  
cout << b.counter;    //3
```

static keyword

In a file

- A static variable denotes a variable whose scope is limited to the file it's in
- Allows different files to use the same variable name without need to worry about violating the ODR

Smart Pointers

Core Idea: Pointer Abstraction

- A pointer can point to valid or invalid memory
- A pointer that points to valid memory can be changed
- Many issues: Invalid Accesses, Memory Leaks

Smart Pointers are **guaranteed to point to valid memory**, and **will clean up resources** when destroyed.

Smart Pointer Summary(simplified)

- Acquire memory in constructor
- Deallocate memory in destructor
- Dereference and access like a raw pointer

```
template<typename T>
class SmartPtr {
public:
    explicit SmartPtr(T *p_in)
        : ptr{ p_in } {}

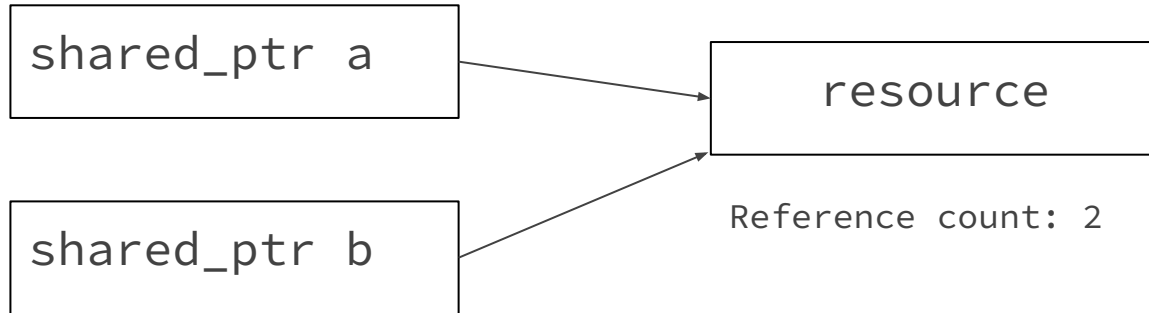
    ~SmartPtr() { delete ptr; }

    T &operator*() { return *ptr; }

private:
    T *ptr;
};
```

shared_ptr<T>

- Multiple shared pointers can point to a resource
- When all shared pointers are destroyed, resource is released
- Maintains a “reference count” for the resource



shared_ptr<T> Example

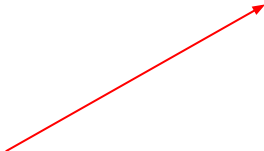
```
{
shared_ptr<int> b;
{
    shared_ptr<int> a = make_shared(10); //resource allocated
    b = a; //copy of shared pointer
}
//a is destroyed
}
//b is destroyed
//resource freed
```

unique_ptr<T>

- Ensures only one owner for a given resource
- **Cannot be copied**, ownership must be transferred from one scope to another using `std::move()`



No other unique ptr



unique_ptr<T> Example

```
{  
unique_ptr<int> b;  
    {  
        unique_ptr<int> a = make_unique(10); //resource allocated  
        b = std::move(a); //must move instead of copy  
    }  
}  
//b is destroyed  
//resource freed
```

Best Practices

— — —

- Use `make_shared` or `make_unique` respectively to avoid working with raw pointers and memory management
- Use `unique_ptr` unless you need to support shared ownership
- Use `unique_ptr` to guarantee single ownership

Read:

https://en.cppreference.com/w/cpp/memory/unique_ptr

https://en.cppreference.com/w/cpp/memory/shared_ptr

Converting Code: Raw Pointers

```
struct big { //expensive to copy
```

```
    ...
```

```
}
```

```
big* a = new big();
```

```
big* b = a;
```

```
delete b; //needed to not leak memory
```

```
//a and b still accessible!
```

```
*a // segfault or worse
```

Converting Code: Smart Pointers

```
struct big { //expensive to copy
    ...
}
shared_ptr<big> a = make_shared<big>();
{
    shared_ptr<big> b = a; //copy
}
//impossible to have invalid access
//resource still cleaned up
```

Questions?